

Personalized AI Financial Guidance System

CS 639: Introduction to Foundation Models — Spring 2026

University of Wisconsin–Madison

Course Instructor: Professor Fred Sala

Team: Fairoz Khan • Aaditya Malhotra • Mohammad Zaidan • Anas Al-Rasbi

Repository: github.com/fairozkhan10/cs639-foundation-models-project

1. Introduction

Large language models (LLMs) have demonstrated impressive capability across a wide range of tasks, yet their utility as personal financial advisors remains limited by a fundamental constraint: they produce advice that is generic rather than individualized. When a user asks, “How can I save more money?”, a vanilla GPT-4o response recommends generic practices such as tracking expenses and cutting subscriptions—advice that is technically correct but disconnected from the user’s actual financial situation. Real financial coaching is effective precisely because it is grounded in the individual: their income, spending habits, debt burden, and concrete goals.

This project investigates whether Retrieval-Augmented Generation (RAG) can bridge this gap. RAG augments LLM generation by injecting retrieved context—in our case, both user-specific financial profile data and a curated financial knowledge base—directly into the prompt at inference time. We hypothesize that this approach will produce advice that is measurably more personalized, relevant, actionable, and grounded than a baseline LLM operating without any retrieved context.

Hypothesis. A RAG-based financial coaching assistant that retrieves user-specific financial context at inference time will produce responses that are more personalized, relevant, and actionable than a baseline LLM without retrieval.

We evaluate this hypothesis through a controlled experiment across 50 synthetic user profiles and three financial coaching questions per profile (150 total evaluation pairs), using GPT-4o as an LLM-as-a-judge scorer across four dimensions: personalization, relevance, actionability, and grounding. All four dimensions show statistically significant improvement under RAG ($p < 0.001$), strongly confirming the hypothesis.

2. Related Work

Retrieval-Augmented Generation was introduced by Lewis et al. (2020), who demonstrated that conditioning generation on retrieved Wikipedia passages significantly improved open-domain question answering accuracy. Subsequent work by Gao et al. (2023) surveying RAG techniques expanded the paradigm to include dense retrieval, hybrid retrieval, and iterative retrieval strategies, establishing RAG as a general-purpose technique for grounding LLM outputs.

In the domain of personalized AI systems, Salemi and Zamani (2024) suggested that prefixing retrieved personal context can be an effective alternative to fine-tuning for personalization tasks. This directly motivates our approach of injecting user profile data into the prompt rather than fine-tuning the underlying model.

For financial AI specifically, prior work has focused primarily on document question answering over earnings reports and regulatory filings (e.g., FinQA by Chen et al., 2021) rather than personalized coaching. Commercial tools such as Copilot Money and Cleo provide budgeting assistance but do not leverage retrieval over a structured user financial profile combined with a domain knowledge base in the way we propose.

On evaluation methodology, Zheng et al. (2023) validated the LLM-as-a-judge paradigm, showing that GPT-4 evaluations correlate strongly with human expert ratings on open-ended generation tasks. We adopt this approach with an independent GPT-4o judge, using randomized response ordering to mitigate position bias, and report paired t-test statistics and Cohen’s d effect sizes to quantify significance and practical magnitude.

3. Methodology

3.1 System Architecture

The system consists of three main components: a data layer (user profiles and knowledge base), a retrieval layer (FAISS vector store), and a generation layer (GPT-4o via LangChain). At inference time, a user question is embedded using OpenAI `text-embedding-3-small` and used to retrieve the top- k most similar knowledge chunks from a FAISS index. A separately retrieved (exact-match) user profile is formatted and prepended to the prompt alongside the knowledge chunks. GPT-4o then generates a grounded, personalized response conditioned on this enriched context. The baseline mode uses the same GPT-4o model but with no retrieved context, providing a clean comparison.

Our contribution comprises the end-to-end pipeline design, the archetype-driven synthetic profile generator, the dual-stage retrieval logic (exact-match profile lookup combined with semantic FAISS retrieval over the curated knowledge base), the prompt templates, and the full evaluation framework (paired comparison harness, judge validation/retry/checkpointing, statistical analysis, and visualization). The underlying foundation models (GPT-4o, `text-embedding-3-small`), the FAISS library, and the LangChain framework are third-party components used as-is.

Table 1: *System configuration and technical specifications.*

Component	Details
Embeddings	OpenAI <code>text-embedding-3-small</code>
Vector Store	FAISS (IndexFlatIP, cosine similarity, local)
LLM	GPT-4o (temperature 0.3, max_tokens 1024)
Framework	LangChain (Python)
Chunking	RecursiveCharacterTextSplitter (size 500, overlap 50)
Retrieval	top_k = 3, similarity threshold = 0.35
Judge Model	GPT-4o (temperature 0, independent invocation)

3.2 Data

User Profiles. We generated 50 synthetic user profiles using the Faker library and a set of six archetype templates: Recent Graduate, Young Professional, High Earner Poor Saver, Frugal Saver, Struggling Middle, and Near-Retiree. Each profile includes 15 spending categories (Rent/Mortgage, Groceries, Dining Out, Transportation, etc.), annual income, total debt with type, emergency fund and retirement account status, savings rate, and 2–4 financial goals. Spending patterns are generated using archetype-specific weights (e.g.,

higher dining and entertainment for “High Earner Poor Saver”), with small per-profile random variation (multiplicative noise sampled from $U[0.7, 1.3]$) to avoid identical profiles and increase realism.

While synthetic, these profiles are designed to approximate realistic financial scenarios; however, they do not capture the full complexity of real-world financial behavior.

Knowledge Base. We curated 17 financial guidance documents drawn from Investopedia, NerdWallet, and general financial planning sources, covering topics including the 50/30/20 budgeting rule, emergency fund basics, debt repayment strategies (snowball vs. avalanche), retirement accounts (401k and IRA), investing basics, student loan repayment, saving for a home down payment, lifestyle inflation, credit score building, and catch-up contributions for ages 50+. Documents were chunked with `RecursiveCharacterTextSplitter` (chunk size 500 tokens, overlap 50 tokens) and embedded using `text-embedding-3-small`.

3.3 Retrieval and Prompt Design

We use FAISS `IndexFlatIP` (inner product on L2-normalized vectors, equivalent to cosine similarity) for retrieval. At query time, we retrieve `top_k = 3` knowledge chunks with a similarity threshold of 0.35; chunks below this floor are dropped. User profile lookup is performed by exact name match, not vector search, to guarantee profile fidelity. The RAG system prompt instructs the model to reference specific numbers from the user’s profile, cite figures from the knowledge documents, and tie recommendations to stated goals. The baseline uses a simpler system prompt with no grounding instructions and receives no retrieved context.

3.4 Evaluation Design

We conducted a controlled comparison between a baseline GPT-4o model and a RAG-enhanced version across 150 evaluation instances (50 profiles $\times$ 3 questions), ensuring identical prompts except for retrieved context.

Each profile is evaluated on three questions (“How can I save more money each month?”, “What should I do about my debt?”, and “Am I on track for retirement?”), yielding 150 evaluation pairs. Each (profile, question) pair produces one RAG response and one baseline response, which are scored independently by a GPT-4o judge on four dimensions (1–5 scale):

- **Personalization:** degree to which the response references the user’s specific financial situation.
- **Relevance:** degree to which the advice directly addresses the question asked.
- **Actionability:** degree to which the advice is concrete and implementable.
- **Grounding:** degree to which claims are supported by verifiable financial knowledge or explicitly grounded in retrieved context (for RAG responses).

To mitigate position bias, the order in which RAG and baseline responses are presented to the judge is randomized per evaluation pair. We report paired t-tests (paired by user-question) and bootstrap 95% confidence intervals, as well as Cohen’s d effect sizes for practical significance.

4. Results and Analysis

4.1 Overall Comparison

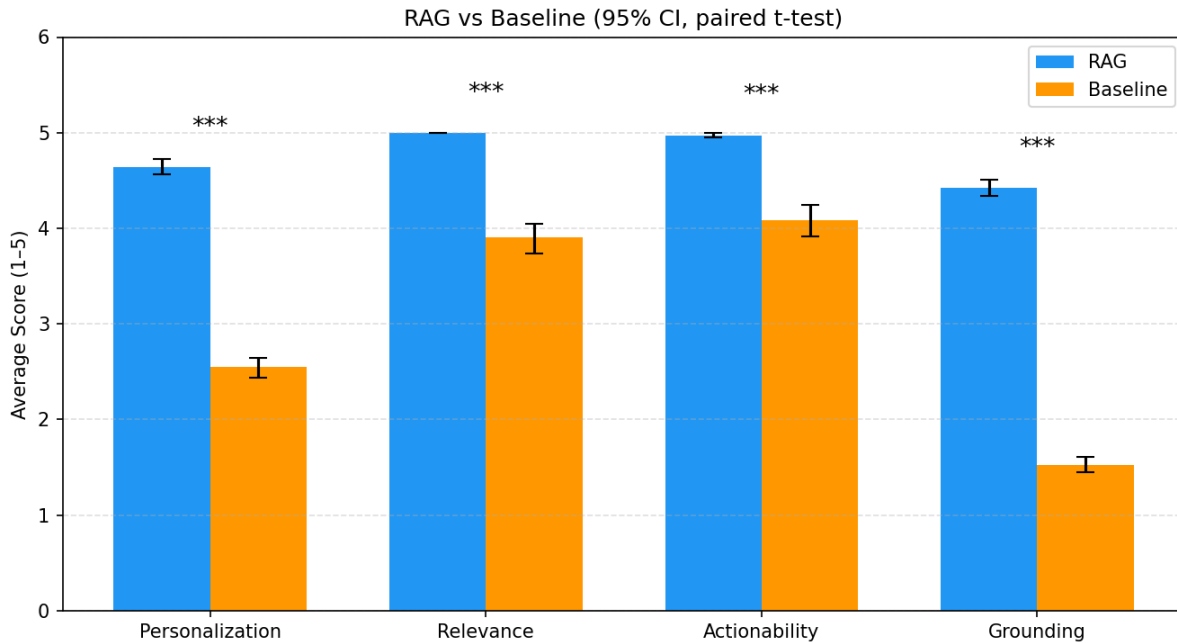
Table 2 summarizes the mean scores, deltas, p-values, and effect sizes across all 150 valid evaluation pairs. RAG outperforms the baseline on all four dimensions with $p < 0.001$, confirming the hypothesis.

Table 2: *RAG vs. Baseline evaluation results (150 pairs, GPT-4o judge). All differences are significant at $p < 0.001$.*

Dimension	RAG	Baseline	Δ	p-value	Cohen's d
Personalization	4.64	2.55	+2.09	< 0.001	3.01
Relevance	5.00	3.91	+1.09	< 0.001	1.11
Actionability	4.97	4.09	+0.89	< 0.001	0.90
Grounding	4.42	1.52	+2.90	< 0.001	5.39

The largest gains are in Grounding ($\Delta = +2.90$, $d = 5.39$) and Personalization ($\Delta = +2.09$, $d = 3.01$), both with very large effect sizes by conventional standards ($d > 0.8$). This pattern is exactly what RAG should produce: grounding improves dramatically because the model now has retrieved knowledge to anchor its claims, and personalization improves because the user's full financial profile is injected directly into the prompt, enabling responses that reference specific income, debt, and spending figures, as illustrated in Section 4.4.

Relevance ($\Delta = +1.09$, $d = 1.11$) and Actionability ($\Delta = +0.89$, $d = 0.90$) also improve significantly, reflecting that advice grounded in specific numbers and supported by retrieved context yields more on-topic and concrete recommendations. RAG approaches ceiling on Relevance (mean 5.00) and Actionability (4.97), while the baseline performs respectably on these dimensions (3.91 and 4.09) since GPT-4o produces structurally sound advice even without retrieval. The decisive separation comes from the two dimensions retrieval directly targets—Grounding and Personalization—where baseline scores fall to 1.52 and 2.55 respectively.

**Figure 1:** *RAG vs. Baseline mean scores with 95% bootstrap confidence intervals (paired t-test, *** = $p < 0.001$).*

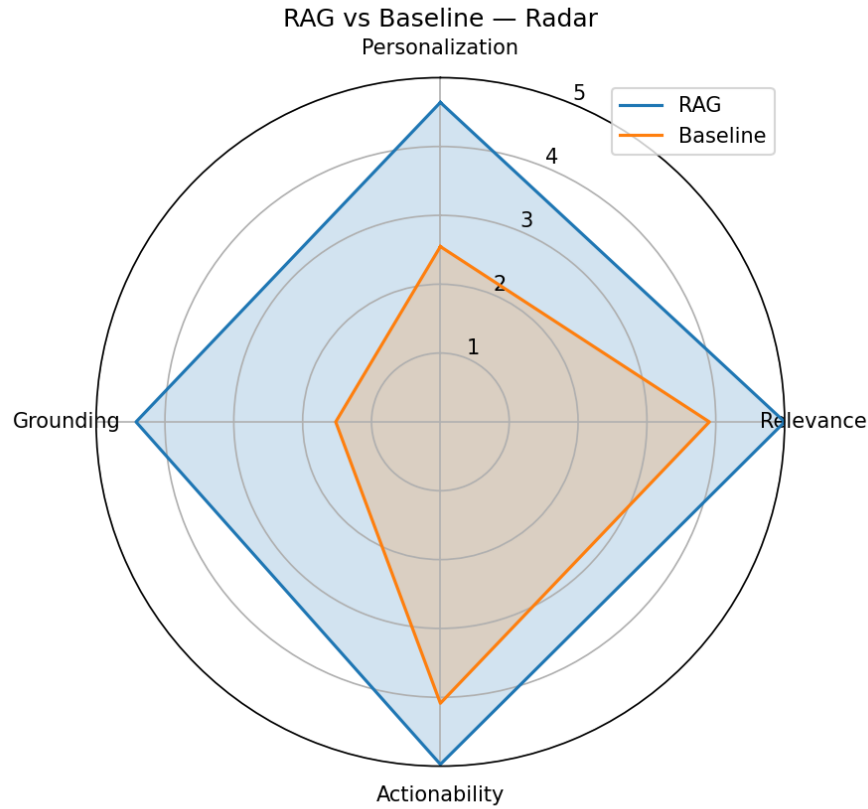


Figure 2: Radar chart showing RAG advantage across all four evaluation dimensions.

4.2 Per-Question Analysis

Figure 3 breaks down scores by question. RAG maintains a consistent advantage across all three questions on all four dimensions. The personalization gap is widest for the debt question ($\Delta \approx +2.46$), where RAG can reference the user's exact debt amount and type while the baseline can only describe generic repayment frameworks. Even the narrowest per-question personalization gap (the retirement question, $\Delta \approx +1.84$) is still very large by conventional effect-size standards.

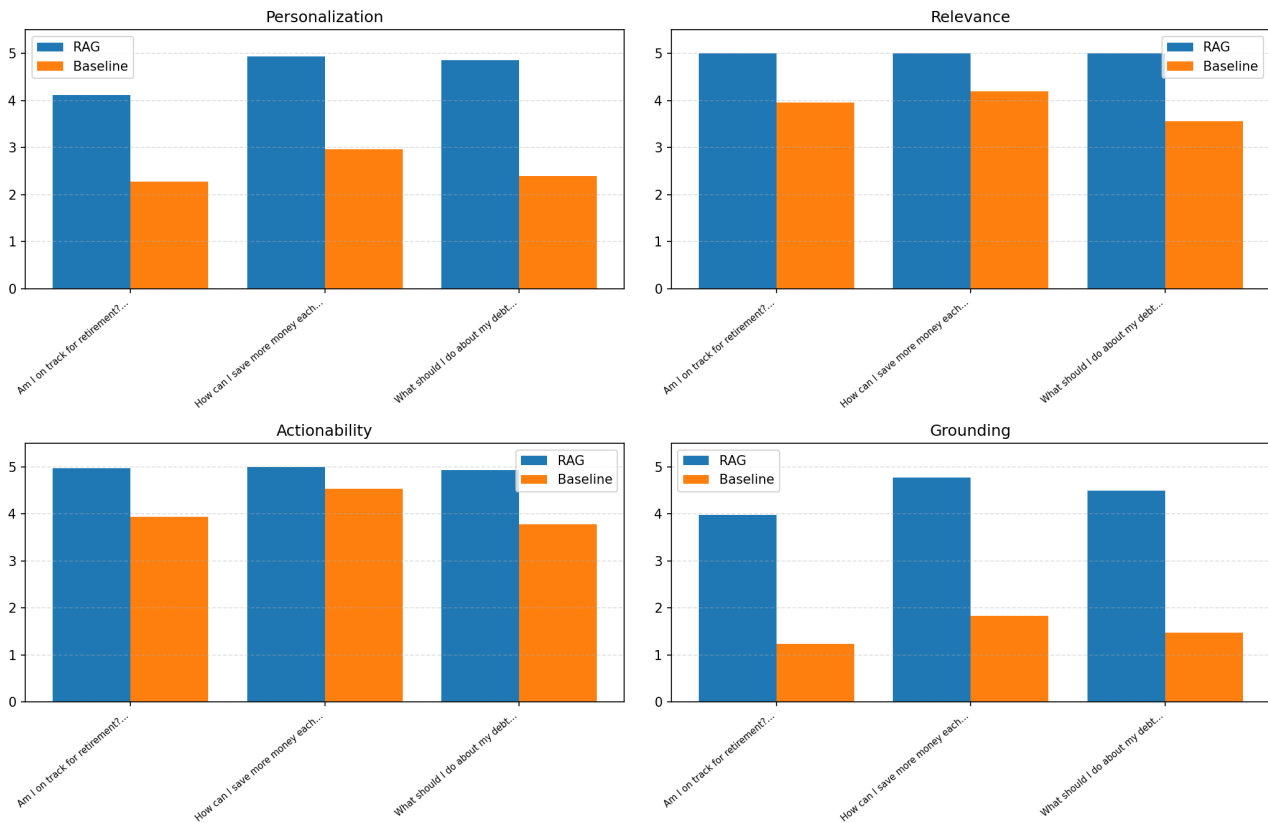


Figure 3: Per-question breakdown of RAG vs. Baseline scores across all four dimensions.

4.3 Per-User Analysis

Figure 4 shows per-user score differences (RAG minus Baseline) across all 50 profiles. Green indicates RAG advantage; red would indicate baseline advantage. RAG outperforms every single user on both Personalization (deltas range from +1.33 to +2.67) and Grounding (every user shows a positive delta of at least +1.0). No user across any of the four dimensions shows a consistent baseline advantage, indicating the RAG improvement is uniform across archetypes rather than driven by a subset of profiles.

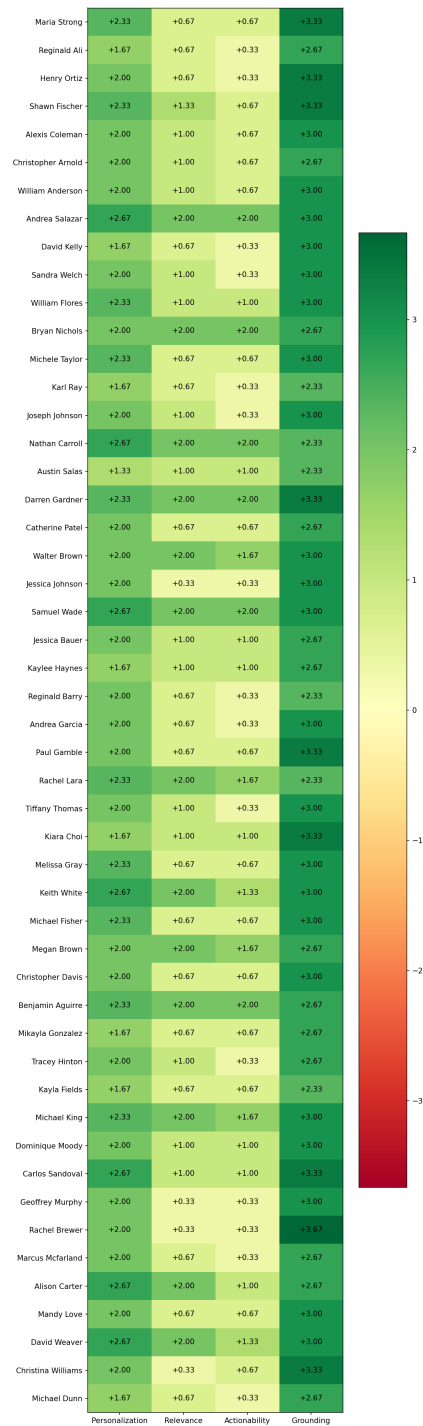


Figure 4: Per-user heatmap of RAG vs. Baseline score differences ($RAG - Baseline$). Green = RAG advantage.

4.4 Qualitative Examples

The following examples illustrate the qualitative difference between RAG and baseline responses for the same question posed to two users.

Example 1 — David Kelly

Profile: High Earner Poor Saver; income \$13,218/month; savings rate -4.9% ; debt \$9,918 car loan; goals: build \$10k investment portfolio + max out 401k.

User Prompt: How can I save more money each month?

RAG Response (excerpt): To improve your savings each month, David, let's focus on a few key areas based on your financial profile and goals. Given your current savings rate of -4.9% , automate a fixed 10% of your income (\$1,321.85) into savings as soon as you receive your paycheck. You currently spend \$1,451.82 monthly on dining out—cutting this in half would save approximately \$725.91. Your subscriptions and entertainment total \$2,358.08 monthly; a 20% reduction would free up \$471.62. To max out your 401k, contributions need to ramp toward the \$22,500 limit. With a \$9,918 car loan, allocate part of these freed funds toward paying it down faster to reduce interest payments. Together these changes move you from a negative savings rate toward your \$10k investment portfolio goal.

Baseline Response (excerpt): Saving more money each month can be achieved by making a few strategic changes. 1. Create a budget—track income and expenses. 2. Set savings goals—emergency fund, vacation, retirement. 3. Pay yourself first—automatic transfers each payday. 4. Reduce unnecessary expenses—dining out less, canceling unused subscriptions. 5. Shop smart, minimize debt, increase income, use high-yield savings accounts. The key to successful saving is consistency and discipline; even small changes can add up over time.

Example 2 — Sandra Welch

Profile: Frugal Saver; income \$4,952/month; savings rate 24.6%; \$4,843 credit card debt; goals: become debt-free + save for a vacation.

User Prompt: What should I do about my debt?

RAG Response (excerpt): Sandra, given your financial profile and goals, here's a tailored plan. You have \$4,843 in credit card debt—given that credit card interest rates can be as high as 24%, the Debt Avalanche method is the best fit. You currently allocate \$646.18 to savings and \$606.72 to investments monthly; redirecting an additional \$300/month toward the credit card could pay it off in approximately 16 months. Maintain minimum payments on any other obligations. Once cleared, redirect those funds back to your \$10k investment portfolio and Roth IRA goals. To prevent recurrence, use your credit card only for expenses you can pay off in full each month.

Baseline Response (excerpt): Dealing with debt can be challenging, but with a structured approach you can eliminate it. 1. Assess your debt—list creditors, amounts, interest rates. 2. Create a budget. 3. Choose a repayment strategy—Snowball (smallest balance first) or Avalanche (highest interest first). 4. Negotiate with creditors for lower rates. 5. Consider consolidation. 6. Increase income. 7. Avoid accumulating more debt. 8. Seek professional help if overwhelmed. Tackling debt takes time and discipline, but with a clear plan you can achieve financial freedom.

The difference is clear: the RAG system produces advice that references the user's exact income, spending category amounts, debt figures, and goals, enabling specific dollar targets and timelines. The baseline produces structurally sound but completely impersonal advice that could apply to any user.

5. Conclusions and Future Work

This project demonstrates that RAG-based retrieval of user-specific financial profiles and domain knowledge significantly improves the quality of LLM-generated financial advice across all four evaluation dimensions.

The hypothesis is strongly confirmed: grounding improves by +2.90 points ($d = 5.39$), personalization by +2.09 ($d = 3.01$), relevance by +1.09 ($d = 1.11$), and actionability by +0.89 ($d = 0.90$), all with $p < 0.001$ across 150 evaluation pairs. The two dimensions retrieval is specifically designed to support—grounding and personalization—show the largest effects, providing direct mechanistic evidence that the gains arise from retrieval rather than incidental prompt differences.

Several important limitations should be noted. First, all user profiles are synthetic; real-world financial data is noisier, more complex (e.g., irregular income, joint accounts, investment portfolios), and more sensitive. The gap between RAG and baseline may differ on real data. Second, the LLM-as-a-judge evaluation, while validated in the literature, may exhibit systematic biases—in particular, favoring longer or more structured responses, which the RAG system tends to produce. Third, our knowledge base contains only 17 curated documents covering common personal-finance topics; performance on questions outside this scope (e.g., complex tax planning, derivatives, estate law) is untested. Fourth, the evaluation uses three coaching questions per profile; broader topical coverage would strengthen external validity.

Future directions include: (1) integrating real anonymized financial data from open banking APIs or Kaggle datasets to validate results beyond synthetic profiles; (2) improving knowledge grounding through explicit citation enforcement in the prompt or chain-of-thought reasoning; (3) exploring iterative or multi-hop retrieval to handle complex questions that require synthesizing multiple knowledge sources; (4) conducting human evaluation studies to validate LLM-as-a-judge scores; and (5) extending the system to support multi-turn coaching sessions where the retrieval index is updated as the conversation evolves.

Overall, this work provides evidence that the RAG paradigm is a practical and effective approach to personalizing LLM-based advisory systems, with strong implications for financial coaching, health coaching, and other domains where individualization is the key differentiator between useful and generic advice.

References

- [1] Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 33.
- [2] Gao, Y., Xiong, Y., Gao, X., et al. (2023). Retrieval-augmented generation for large language models: A survey. *arXiv:2312.10997*.
- [3] Salemi, A., & Zamani, H. (2024). Evaluating retrieval quality in retrieval-augmented generation. *Proceedings of SIGIR 2024*.
- [4] Chen, Z., Chen, W., Smiley, C., et al. (2021). FinQA: A dataset of numerical reasoning over financial data. *Proceedings of EMNLP 2021*.
- [5] Zheng, L., Chiang, W.-L., Sheng, Y., et al. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *Advances in Neural Information Processing Systems (NeurIPS)*, 36.
- [6] Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3), 535–547.
- [7] Chase, H. (2022). LangChain [Computer software]. <https://github.com/langchain-ai/langchain>
- [8] OpenAI. (2024). GPT-4o System Card. <https://openai.com/index/gpt-4o-system-card/>
- [9] OpenAI. (2024). New embedding models and API updates [text-embedding-3-small]. <https://openai.com/index/new-embedding-models-and-api-updates/>
- [10] Faraglia, D., & Other Contributors. (2024). Faker [Python library]. <https://github.com/joke2k/faker>